Expression Conversion Cheat Sheet

Operator Precedence

```
^       -> 3
* /     -> 2
+ -     -> 1
```

Associativity

```
+ - * /  -> Left Associative
^          -> Right Associative
```

Example:

```
A-B-C = (A-B)-C
A^B^C = A^(B^C)
```

Infix → Postfix

Rules

1. Operand → Add to answer
2. '(' → Push
3. ')' → Pop till '('

4. Operator:

5. Pop operators with higher precedence

6. For same precedence:
    - Pop for + - * /
    - Don't pop for ^

Condition

```
while(!st.empty() &&
      (prec(st.top()) > prec(ch) ||
      (prec(st.top()) == prec(ch) && ch != '^')))
```

Memory

```
Operand -> Output
(       -> Push
)       -> Pop till (
Operator-> Pop stronger operators then push
End     -> Pop all
```

Infix → Prefix

Steps

```
1. Reverse expression
2. Swap ( and )
3. Convert to Postfix
4. Reverse answer
```

Special ^ Condition

```
while(!st.empty() &&
      (prec(st.top()) > prec(ch) ||
      (prec(st.top()) == prec(ch) && ch == '^')))
```

Why?

```
Postfix : equal ^ -> DON'T POP
Prefix  : equal ^ -> POP
```

Prefix → Infix

Direction

```
Right -> Left
```

Rules

Operand:

```
push(string(1,ch));
```

Operator:

```
op1 = st.top(); st.pop();
op2 = st.top(); st.pop();

temp = "(" + op1 + ch + op2 + ")";
st.push(temp);
```

Formula

```
(op1 operator op2)
```

Example:

```
+AB -> (A+B)
```

Prefix → Postfix

Direction

```
Right -> Left
```

Rules

Operand:

```
push(string(1,ch));
```

Operator:

```
op1 = st.top(); st.pop();
op2 = st.top(); st.pop();

temp = op1 + op2 + ch;
st.push(temp);
```

Formula

```
op1 op2 operator
```

Example:

```
+AB -> AB+
```

Postfix → Infix

Direction

```
Left -> Right
```

Rules

Operand:

```
push(string(1,ch));
```

Operator:

```
op2 = st.top(); st.pop();
op1 = st.top(); st.pop();

temp = "(" + op1 + ch + op2 + ")";
st.push(temp);
```

Formula

```
(op1 operator op2)
```

Example:

```
AB+ -> (A+B)
```

Postfix → Prefix

Direction

```
Left -> Right
```

Rules

Operand:

```
push(string(1,ch));
```

Operator:

```
op2 = st.top(); st.pop();
op1 = st.top(); st.pop();

temp = ch + op1 + op2;
st.push(temp);
```

Formula

```
operator op1 op2
```

Example:

```
AB+ -> +AB
```

Universal Pop Order Trick

Prefix Conversions

Scanning Right → Left

```
op1 = st.top(); st.pop();
op2 = st.top(); st.pop();
```

Use:

```
op1 first
op2 second
```

Postfix Conversions

Scanning Left → Right

```
op2 = st.top(); st.pop();
op1 = st.top(); st.pop();
```

Use:

```
op1 first
op2 second
```

Remember:

First pop = RIGHT operand Second pop = LEFT operand

```
---

# One-Page Revision Table

| Conversion | Scan Direction | Build Formula |
|------------|---------------|---------------|
| Infix → Postfix | Left → Right | Operator Stack |
| Infix → Prefix | Reverse → Postfix → Reverse | Operator Stack |
| Prefix → Infix | Right → Left | (op1 op op2) |
| Prefix → Postfix | Right → Left | op1 op2 op |
| Postfix → Infix | Left → Right | (op1 op op2) |
| Postfix → Prefix | Left → Right | op op1 op2 |

---

# Golden Rules

```text

Prefix  -> Scan Right → Left
Postfix -> Scan Left → Right

Prefix:
    pop op1
    pop op2
```
```

```
Postfix:
    pop op2
    pop op1
```

```
Postfix : equal ^ -> DON'T POP
Prefix  : equal ^ -> POP
```